ST458 Assignment (Deadline: 13 Apr 2026 2pm)

Tengyao Wang

2026-03-03

Objective

In this assignment, you will develop a trading strategy using historical market data from a synthetic financial market. Your goal is to design an algorithm that effectively identifies trading opportunities and maximises wealth over the test period.

Dataset

The investment universe consists of 100 synthetic ETFs. You are provided with four years of historical training data on Moodle (`df_train.csv`) in the form of a dataframe with the following columns:

- `date` – Trading date
- `symbol` – Name of the ETF
- `open, close, low, high` – Opening, closing, low, and high prices for the trading day
- `volume` - trading volume on the day

In real financial markets, profitable trading opportunities are scarce, especially in low- to mid-frequency data. To make this dataset more suitable for strategy development, a small amount of predictive signal has been injected within the data. Your challenge is to identify and capitalise on this signal.

Your Task

Using the training data, you should develop a trading algorithm and **submit the following**:

1. A well-structured **PDF report** of at most 3 pages, named `GroupX.pdf` (where X is your group identifier), explaining:

- Data preparation: Any preprocessing or feature engineering performed,
- Model development: Statistical or machine learning models considered and tested,
- Final strategy: The chosen trading algorithm and rationale behind its design.

2. An **R script**, named `GroupX.R` , including the following **two functions**:

i. `initialise_state(data)`

   - Input: The full training dataset (`data`).
   - Output: An R object/list (`state`) that stores any precomputed information needed for future trading decisions.

ii. `trading_algorithm(new_data, state)`

   - Inputs: `new_data` – Market data for the next trading day, and `state` – the updated state from previous trading decisions.
   - Outputs: A named vector of `trades` (which ETFs to buy/sell and in what **dollar quantity**), and an updated `new_state` object for use in future trades.

3. Auxiliary files (optional)
    - You may submit auxiliary files (e.g., saved models, preprocessed data) that your R script can load and use.

Details

- You may assume that you start with 1 unit of wealth at the beginning of the test period, with no initial position in all ETFs.
- You may assume that the trades are executed right before the closing of each trading day, at the closing price.
- Note that the output `trades` from `trading_algorithm` is measured in **amount of wealth traded per asset**
- You should try your best to avoid your wealth decrease below 0 (see performance metric below). If your wealth drops below 0, it remains at 0 for the rest of the test period.
- You may assume there are no missing data in both the training and test data frames.
- A transaction cost of 5 basis points (0.05%) is charged on the absolute dollar value of each trade. Fractional positions in each ETF are permitted.
- Example R code implementing a simple momentum strategy (`example_script.R`) is available on Moodle to help you structure your submission correctly. Your trading algorithm will be evaluated via walk-forward backtesting using `walk_forward.R` , which is also available on Moodle. Please use it to **check that your code works properly** before submission.
- It is possible to submit your code in Python instead of R. However, as the course is taught in R, and to consistency of learning for all group members, you should only choose Python if all group members agree to do so. If your group decides to proceed with Python, please email me to request approval and copy all group members on the message.

Performance metric

- Your trading algorithm will be tested on 3 years of future synthetic data.
- There are 100 different realisations of this future data. Your final score is based on the average log wealth at the end of the test period across these 100 simulations.
- To avoid have $-\infty$ for zero wealth at the end, the smallest log wealth is defined to be -10. Note that this is still a heavy penalty.
- Time limit for processing (one realisation of) 3 years' test data is **10 minutes**. Code timing is done on a desktop machine with 8 cores, each at 2.9GHz. This time limit should be more than sufficient. Your algorithm is terminated after 10 minutes and each additional day untraded will be treated as incurring a -0.01 loss in log wealth (i.e. a 1% loss).

Marking criteria

The assignment has 100 marks in total and allocated as follows.

Category	Weighting	Evaluation Criteria
1. Quality of Coding	20%	Well-documented, clear, and logically structured code.

Category	Weighting	Evaluation Criteria
2. Algorithm Performance	30%	Your trading algorithm will be tested on 100 different realisations of 3 years of future synthetic data (see the 'Performance metric' section above). Your score is based on the average log wealth at the end of the test period across these 100 simulations. The highest-performing team receives 30 marks, and a log return of 0 receives 15 marks (linear scaling in between).
3. Training Process Description	35%	Quality of explanation for data processing, feature selection, and model experimentation.
4. Final Strategy Explanation	15%	Clarity of reasoning behind the chosen strategy.